PROJECT DOCUMENTATION

COMPARISON OF TWO GENERAL TREES

C++ | Data Structures and Algorithms Mini Project

Project Type	DSA Mini Project
Programming Language	C++
Data Structure	General Tree
Core Operation	Recursive structure and value comparison

Add student name, roll number, class, and college details before submission.

CONTENTS

- 1. Abstract
- 2. Introduction
- 3. Problem Statement
- 4. Objectives
- 5. Scope
- 6. Hardware and Software Requirements
- 7. General Tree Representation
- 8. System Design and Working
- 9. Algorithm
- 10. Pseudocode
- 11. C++ Implementation Overview
- 12. Sample Input and Output
- 13. Time and Space Complexity
- 14. Advantages
- 15. Limitations and Future Enhancements
- 16. Test Cases
- 17. Conclusion
- 18. References

1. Abstract

This project constructs two general trees and checks whether they have identical structures and corresponding node values. Each node stores a string value and a list of child pointers. The comparison is performed recursively: node values and child counts are checked first, then corresponding children are compared. The program reports whether the trees are identical.

2. Introduction

A general tree is a hierarchical, non-linear data structure in which each node may have any number of children. Tree comparison is useful for checking whether two hierarchical models match. This project demonstrates tree construction, recursion, traversal, validation, and dynamic memory management in C++.

3. Problem Statement

Write a C++ program to construct two general trees and determine whether they have the same structure and corresponding node values. The program reads node values and parent-to-child relationships, validates each tree, and recursively compares the trees from their roots.

4. Objectives

- Construct two general trees from user input.
- Store a value and child list in each node.
- Validate that each input represents a tree.
- Compare corresponding values and child counts.
- Use recursion to compare all corresponding subtrees.
- Display a clear identical/different result.

5. Scope

The program supports ordered general trees. Children are compared in the order entered. Node values are single tokens. The application is intended for academic demonstration and can be extended to support unordered children or richer node data.

6. Hardware and Software Requirements

- Desktop or laptop computer.
- Windows, Linux, or macOS.
- C++11 or later compiler (GCC/MinGW or equivalent).
- Text editor or IDE such as VS Code or Code::Blocks.

7. General Tree Representation

Each node contains a string value and a vector of pointers to its children. Node 1 is treated as the root. For a valid tree, the root has no parent, every other node has exactly one parent, every node is reachable from the root, and there are no cycles.

```
Example representation: A → children [B, C]; B → child [D]; C and D are leaves.
```

8. System Design and Working

- Read node values and child IDs for Tree 1.
- Validate Tree 1 and repeat construction for Tree 2.

- Compare the roots. If one tree is empty and the other is not, they differ.
- If node values or child counts differ, return false.
- Otherwise compare each pair of children at the same position recursively.
- If every recursive comparison succeeds, report that the trees are identical.

9. Algorithm

1. Start.

2. Read the number of nodes for Tree 1 and create a node for each input value.

3. Read the child IDs for every node; treat node 1 as the root.

4. Validate that Tree 1 has one root, each other node has exactly one parent, all nodes are reachable, and there are no cycles.

5. Repeat Steps 2–4 to construct and validate Tree 2.

6. Call SameTree(root1, root2).

7. If both pointers are NULL, return true. If only one is NULL, return false.

8. Compare the values of the two current nodes. If different, return false.

9. Compare the number of children. If different, return false.

10. For each child position i, recursively compare child i of Tree 1 with child i of Tree 2. If any comparison is false, return false.

11. If all corresponding child comparisons succeed, return true.

12. Display the comparison result, release allocated memory, and end.

10. Pseudocode

```
SAME_TREE(A, B)
  IF A is NULL OR B is NULL
      RETURN (A and B are both NULL)
  IF A.value != B.value
      RETURN false
  IF childCount(A) != childCount(B)
      RETURN false
  FOR each child position i
      IF SAME_TREE(A.child[i], B.child[i]) is false
          RETURN false
  RETURN true
```

11. C++ Implementation Overview

The Node structure stores a value and a vector of child pointers. The createTree function reads the nodes and parent-to-child relationships, validates the tree, and returns the root. The sameTree function recursively checks whether two trees match. The deleteTree function releases allocated nodes after comparison.

12. Sample Input and Output

Example 1: Identical trees

```
Tree 1: A has children B and C; B has child D; C and D are leaves.
Tree 2: A has children B and C; B has child D; C and D are leaves.
Output: Both trees have the same structure and corresponding node values.
```

Example 2: Different trees

```
Tree 1 root value: A
Tree 2 root value: X
Output: The trees are different in structure or corresponding node values.
```

13. Time and Space Complexity

Let n be the number of nodes in each tree. The recursive comparison visits each corresponding node at most once, so comparison takes O(n) time when the trees have the same size. If a mismatch occurs early, it may finish sooner. The recursion uses O(h) stack space, where h is the height of the tree; storing the trees themselves requires O(n) space.

14. Advantages

- Works for general trees with any number of children.
- Recursive comparison is straightforward and readable.
- Checks both structure and node values, not just node count.
- Demonstrates practical recursion and pointer use.

15. Limitations and Future Enhancements

Limitations

- Child order matters in this implementation.
- Node values are read as single words without spaces.
- Very deep trees may cause deep recursive calls.

Future Enhancements

- Support unordered-child comparison using matching or sorting.
- Display both trees graphically.
- Allow multi-word node values.
- Add menu options for editing and displaying tree traversals.

16. Test Cases

Test	Condition	Expected Result
1	Same values and same child order	Identical
2	Different root values	Different
3	Same values but different child count	Different
4	Different value in a leaf node	Different
5	Same nodes but swapped child order	Different in this ordered-tree program
6	Invalid parent relationships or cycle	Tree rejected

17. Conclusion

The Comparison of Two General Trees project uses recursion to compare two hierarchical structures. By checking corresponding node values and child counts at every level, it determines whether the trees match in both structure and data. The project reinforces general-tree representation, recursive algorithms, input validation, and dynamic memory management.

18. References

- C++ reference: https://en.cppreference.com/
- Data Structures and Algorithms course notes on general trees and recursion.
- Project source code and algorithm prepared for this academic mini project.